

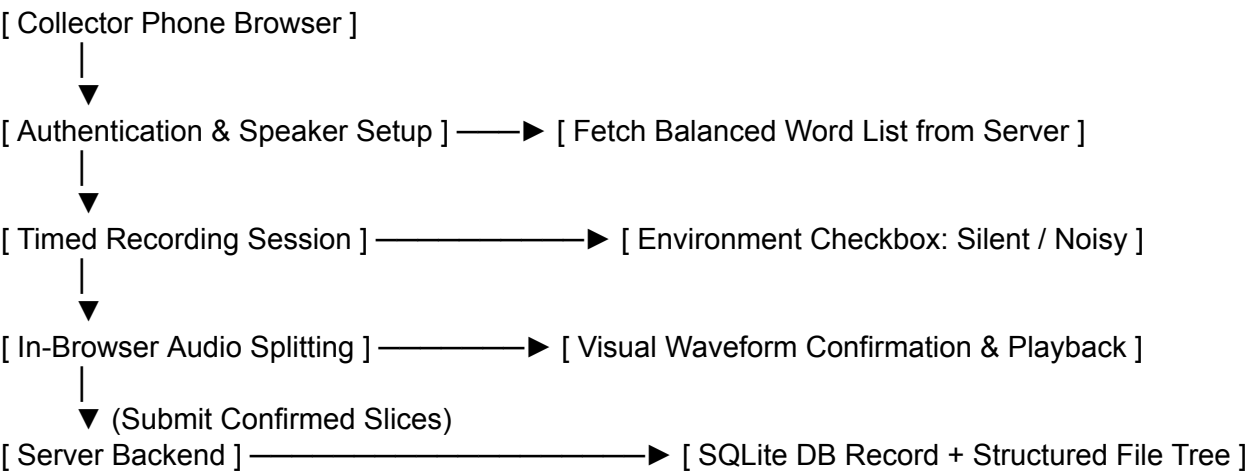
Data Engineering & Collection Tooling

Lead Specification

Role Title: Data Engineering, Synthetic Generation & Acoustic Augmentation Lead **Objective:** Build the complete audio collection web application, implement synthetic dataset generation via Text-to-Speech (TTS), and create an automated audio noise-augmentation pipeline to make the machine learning model resilient in real-world noisy environments. **Tech Stack:** Python (FastAPI / Flask, pydub, librosa, soundfile), Web Frontend (HTML5 Audio API, WaveSurfer.js / Web Audio API), SQLite / CSV Metadata Engine, Open-Source TTS (Piper / Edge-TTS).

Part 1: Audio Collection Web App (Frontend + Backend)

1.1 Architectural Flow



1.2 Recording Protocol & Timing Schedule

When a collector opens the web app and selects/creates a speaker profile, the app conducts a guided multi-step recording session:

Step	Word Type	Duration	Description & Instructions
Step 1	Trigger Word	20 Seconds	Speaker repeats the trigger word multiple times with 1-second pauses, varying pitch, speed, and tone (whisper, assertive, normal).

Step	Word Type	Duration	Description & Instructions
Step 2	Rhyming Word 1	5 Seconds	Speaker repeats a phonetically similar word (e.g., if trigger is "Suraksha", say "Surakshit" or "Raksha").
Step 3	Rhyming Word 2	5 Seconds	Speaker repeats a second balanced rhyming word.
Step 4	Negative Word 1	5 Seconds	Speaker repeats a random everyday word (e.g., "Table", "Fan", "Water").
Step 5	Negative Word 2	5 Seconds	Speaker repeats a second random everyday word.

1.3 Server-Side Word Balancing Engine

The backend reads a master file (words_config.json or words.csv) containing:

- trigger_words: ["suraksha"]
- rhyming_words: ["surakshit", "raksha", "surplus", "akasha"]
- negative_words: ["laptop", "bottle", "window", "light", "metro", "college", "charger", "campus"]

Balancing Logic:

- Every new session automatically receives the trigger word.
- The server tracks how many times each rhyming word has been recorded across all speakers and picks the **two least recorded rhyming words** to ensure equal distribution.
- The server randomly picks **two negative words** from the negative list.

1.4 Web App Frontend Specifications

- **Auth Screen:** Simple PIN / Password gate so only the 6 authorized team members can upload data.
- **Speaker Selection:** Dropdown of existing speakers or an "+ Add New Speaker" input field.
- **Environment Tag:** A prominent checkbox on the recording screen:
 - ☐ Noisy Room / Background Chatter (Checked = Noisy, Unchecked = Silent Room).
- **Live Recording Interface:**
 - Displays the target word prominently with a live countdown timer.
 - Live visualizer showing microphone input volume.
- **Smart Audio Slice & Confirm UI:**
 - Once the timer ends, the Web Audio API analyzes the recorded waveform and detects pauses (silence threshold: volume < -35 dB for > 300 ms).
 - The app automatically slices the recording into individual ~1.0-second word clips.

- The UI displays a card for each extracted chunk with an inline play button and a [x] Keep / Discard toggle.
- The user taps **"Confirm & Upload All Chunks"** to send the verified audio files to the server.

1.5 Storage & Database Hierarchy

Every uploaded audio slice is normalized to **16 kHz, 16-bit Mono .wav** and saved into the following directory tree:

```
dataset/
├── trigger_word/
│   ├── rahul/
│   │   ├── rahul_suraksha_silent_001.wav
│   │   └── rahul_suraksha_noisy_002.wav
│   └── priya/
│       └── priya_suraksha_silent_001.wav
├── negative_word/
│   ├── rhyming/
│   │   ├── rahul/
│   │   │   └── rahul_raksha_silent_001.wav
│   │   └── priya/
│   │       └── priya_surakshit_noisy_001.wav
│   └── general/
│       ├── rahul/
│       │   └── rahul_laptop_silent_001.wav
│       └── priya/
│           └── priya_bottle_noisy_001.wav
```

Metadata Database Schema (metadata.db / dataset_catalog.csv):

Field	Type	Description
file_id	String	Unique file identifier (UUID)
file_path	String	Relative disk path
speaker_name	String	Name of the person who spoke
word_spoken	String	The actual word spoken (e.g., "suraksha")
category	String	trigger_word, rhyming_word, or general_negative
environment	String	silent_room or noisy_environment
duration_ms	Integer	Length of the audio chunk in milliseconds
created_at	Timestamp	Date and time of upload

Part 2: Synthetic Dataset Generation via TTS

To bootstrap thousands of diverse training samples without recording thousands of manual

takes, the developer will write a Python pipeline using open-source Text-to-Speech engines (such as **Piper TTS** or **Edge-TTS**).

2.1 Parameter Variation Matrix

The generation script must programmatically iterate through multiple vocal profiles:

- **Voice Profiles:** At least 10 distinct voices (Male, Female, Indian-English, Hindi-accented).
- **Speed/Rate Variations:** 0.75x, 0.90x, 1.0x, 1.10x, 1.25x.
- **Pitch Adjustments:** -15 Hz, -5 Hz, 0 Hz, +10 Hz, +20 Hz.
- **Volume Gains:** -6 dB, 0 dB, +4 dB.

2.2 Synthetic Generation Workflow

1. Read the word lists (Trigger, Rhyming, Negative).
2. Generate 1,500 synthetic .wav files of the custom keyword across all voice, pitch, and speed combinations.
3. Generate 1,000 synthetic .wav files of rhyming and negative words.
4. Auto-trim leading and trailing silence to enforce strict 1.0-second .wav files.

Part 3: Noise Augmentation & Mixing Pipeline

3.1 Why the "Clap" Test Failed

When an AI model is trained only on clean, silent-room audio, its convolutional filters learn that speech features are strictly surrounded by flat, zero-energy frequencies.

When someone claps, drops a book, or turns on a fan:

1. **Spectrogram Masking:** The high-frequency impulse of a clap draws a solid vertical line across the entire Mel-spectrogram.
2. **Feature Distortion:** This energy line cuts through the custom keyword's delicate acoustic patterns, causing the neural network's confidence to drop from 95% down to 60%.

To make the model robust against claps, fans, and room echoes, you must **mix noise directly into the clean audio during dataset preparation**.

3.2 Automated Noise Merging Pipeline

1. **Source Noise Datasets:** Download open-source noise banks from the **Google Speech Commands Background Noise** dataset or **ESC-50** (containing claps, ambient chatter, keyboard typing, fan hums, and door knocks).
2. **Dynamic SNR Mixing:** Write a Python script using pydub / librosa that takes every clean recording (human and synthetic) and generates multiple augmented variations at varying **Signal-to-Noise Ratios (SNR)**:
 - **Clean Copy (No Noise):** 100% clean voice.
 - **Low Noise (15 dB SNR):** Voice with subtle background fan hum.
 - **Medium Noise (10 dB SNR):** Voice with ambient room chatter / typing.
 - **High Noise / Impulse (5 dB SNR):** Voice with sudden transient claps or knocks mixed in.

[Clean Audio Clip] ———▶ Clean Sample (Original)

- └─▶ Mixed with Fan Hum @ 15dB SNR
- └─▶ Mixed with Room Chatter @ 10dB SNR
- └─▶ Mixed with Sudden Clap @ 5dB SNR

Part 4: Developer TODO List & Deliverables

Sprint 1: Web Application (Frontend & Backend)

- [] Initialize a FastAPI backend serving HTML5/JS frontend.
- [] Implement user authentication (PIN / Session token).
- [] Create words_config.json and build the server-side balancing algorithm to serve trigger, rhyming, and negative words.
- [] Build the web recorder with live audio capture and countdown timer.
- [] Implement client-side silence detection (pause detection) to auto-split multi-word recordings into 1.0-second cards.
- [] Build the visual confirmation card UI with playback and keep/discard toggles.
- [] Create the server upload endpoint that writes .wav files into the nested folder hierarchy and logs metadata to SQLite.

Sprint 2: Synthetic TTS Generation Pipeline

- [] Write generate_synthetic_data.py utilizing edge-tts or piper-tts.
- [] Iterate through multiple voice profiles, pitch parameters, and speed rates.
- [] Output 1,500 standardized 16 kHz Mono .wav samples into dataset/synthetic/.

Sprint 3: Noise Augmentation & Dataset Packaging

- [] Download background noise and impulse sounds (claps, taps, white noise).
- [] Write augment_dataset.py using pydub to mix clean samples with noise at 5 dB, 10 dB, and 15 dB SNR.
- [] Generate the final structured dataset directory (dataset/final_train/) containing balanced, noise-augmented folders ready for the ML Lead to train the model.

Part 5: Reference Code Templates

1. Word Balancing Backend Logic (server_words.py)

```
import json
import random
from collections import defaultdict

with open("words_config.json", "r") as f:
    CONFIG = json.load(f)

# In-memory or DB tracking of rhyming word counts
rhyming_counts = defaultdict(int)
```

```

def get_words_for_speaker(speaker_name: str) -> dict:
    trigger = CONFIG["trigger_word"]

    # Pick the 2 least recorded rhyming words for balance
    sorted_rhymes = sorted(CONFIG["rhyming_words"], key=lambda w: rhyming_counts[w])
    selected_rhymes = sorted_rhymes[:2]
    for w in selected_rhymes:
        rhyming_counts[w] += 1

    # Pick 2 random negative words
    selected_negatives = random.sample(CONFIG["negative_words"], 2)

    return {
        "speaker": speaker_name,
        "trigger": trigger,
        "rhyming_words": selected_rhymes,
        "negative_words": selected_negatives
    }

```

2. Audio Noise Augmentation Script (augment_dataset.py)

```

import os
import random
from pydub import AudioSegment

def overlay_noise(clean_path: str, noise_path: str, output_path: str, snr_gain: int = -12):
    """
    Overlays background noise (or claps) onto clean voice audio.
    snr_gain: negative value reduces noise volume relative to speech.
    """
    clean_audio = AudioSegment.from_wav(clean_path)
    noise_audio = AudioSegment.from_wav(noise_path)

    # Loop noise if it is shorter than the speech clip
    while len(noise_audio) < len(clean_audio):
        noise_audio += noise_audio
    noise_audio = noise_audio[:len(clean_audio)]

    # Adjust noise volume to target SNR
    adjusted_noise = noise_audio + snr_gain

    # Overlay noise and voice
    augmented = clean_audio.overlay(adjusted_noise)
    augmented.export(output_path, format="wav")

def batch_augment(clean_dir: str, noise_dir: str, output_dir: str):

```

```

os.makedirs(output_dir, exist_ok=True)
noise_files = [os.path.join(noise_dir, f) for f in os.listdir(noise_dir) if f.endswith(".wav")]

for file in os.listdir(clean_dir):
    if not file.endswith(".wav"):
        continue
    clean_file = os.path.join(clean_dir, file)

    # 1. Save original clean file
    clean_audio = AudioSegment.from_wav(clean_file)
    clean_audio.export(os.path.join(output_dir, file), format="wav")

    # 2. Generate noise-augmented version
    selected_noise = random.choice(noise_files)
    aug_name = file.replace(".wav", "_noisy.wav")
    overlay_noise(clean_file, selected_noise, os.path.join(output_dir, aug_name),
snr_gain=-10)

print("[INFO] Augmentation pipeline ready.")

```

3. Synthetic TTS Generation Script (generate_tts.py)

```

import asyncio
import os
import edge_tts

VOICES = [
    "en-IN-NeerjaNeural",
    "en-IN-PrabhatNeural",
    "en-US-GuyNeural",
    "en-US-JennyNeural"
]

RATES = ["-15%", "-5%", "+0%", "+10%", "+20%"]
PITCHES = ["-10Hz", "+0Hz", "+15Hz"]

async def generate_samples(word: str, output_folder: str):
    os.makedirs(output_folder, exist_ok=True)
    idx = 0
    for voice in VOICES:
        for rate in RATES:
            for pitch in PITCHES:
                filename = os.path.join(output_folder, f"syn_{word}_{idx:04d}.mp3")
                communicate = edge_tts.Communicate(
                    text=word,
                    voice=voice,
                    rate=rate,

```

```
        pitch=pitch
    )
    await communicate.save(filename)
    idx += 1
    print(f"[SUCCESS] Generated {idx} synthetic samples for '{word}'.")

if __name__ == "__main__":
    asyncio.run(generate_samples("Suraksha", "dataset/synthetic/trigger/"))
```